



Bahria University Islamabad

Department of Information Technology

PROJECT PROPOSAL

Banking Management System using EMU8086 Assembly Language

Submitted By: Faisal Amin

Enrollment No: 01-135232-019

Submitted By: Muhammad Huzaifa

Enrollment No: 01-135232-058

Program: BS Information Technology

Class: BSIT 6B

Course Title: COAL (Computer Organization and Assembly Language)

Instructor: Adnan Jelani

Submission Year: 2026

Contents

Abstract	3
1 Introduction	4
2 Problem Statement	5
3 Objectives	6
4 Project Title	7
5 Project Scope	8
5.1 Included Features	8
5.2 Limitations	8
6 Core Features	9
6.1 User Features	9
6.2 System-Level Features	9
7 Technical Design in EMU8086	10
7.1 Memory Model	10
7.2 Memory Variables	10
7.3 Interrupts Used	10
7.4 Conversion Techniques	11
8 Environment	12
9 Methodology	13
9.1 Phase 1: Requirement Analysis	13
9.2 Phase 2: System Design	13
9.3 Phase 3: Memory Planning	13
9.4 Phase 4: Coding	13

9.5	Phase 5: Testing	13
9.6	Phase 6: Final Implementation	14
10	Program Structure	15
10.1	Main Menu	15
10.2	Procedure Structure	15
11	Expected Output	16
12	Challenges and Solutions	17
13	Learning Outcomes	18
14	Future Enhancements	19
15	Conclusion	20
	References	21
	Appendix: Code Structure	22

Abstract

The purpose of this project is to design and develop a comprehensive Banking Management System using EMU8086 Assembly Language. The project focuses on creating a secure, efficient, and user-friendly platform that allows users to manage banking operations at the microprocessor level. The system emphasizes data integrity, secure authentication, and transaction processing using assembly-level instructions.

The proposed system includes user account creation, PIN-based login system, deposit and withdrawal operations, balance inquiry, and account details display. It utilizes the EMU8086 emulator environment to demonstrate assembly language concepts. Security is implemented through PIN verification and transaction validation using conditional jumps.

The implementation involves memory variables in the data segment, register manipulation for arithmetic operations, and DOS interrupts for input/output operations. This project demonstrates how low-level programming can solve real-world problems while providing practical experience with microprocessor architecture.

1 Introduction

Technology has transformed the way banks handle customer records and transactions. Modern banking systems allow users to manage accounts, transfer money, and check balances digitally. However, understanding how these systems work at the lowest level—the microprocessor level—is crucial for computer science students. This project develops a simple Banking Management System using EMU8086 Assembly Language.

By working with registers, memory addresses, and interrupts, students gain insight into the underlying mechanisms that power modern financial software. This project helps in understanding low-level programming concepts such as memory management, registers, loops, conditional statements, procedures, stack operations, and interrupt-based input/output operations.

The system simulates basic banking operations through a console-based interface using BIOS and DOS interrupts. The project is designed for the COAL course and focuses on applying EMU8086 Assembly Language concepts in a practical application.

2 Problem Statement

Traditional manual banking systems are slow and difficult to manage. Keeping records on paper causes errors, data loss, and difficulty in updating customer information. From a programmer's perspective, creating a financial system in assembly language presents challenges such as preventing overflow during arithmetic operations, ensuring data integrity in memory, handling ASCII to binary conversions, and implementing secure login mechanisms.

Many educational projects focus on high-level languages but fail to address low-level programming concepts. Students struggle to understand registers, interrupts, and memory management at the processor level. This project addresses the need for a practical application that demonstrates these concepts through banking operations.

The proposed system provides a digital solution where users can create accounts, deposit money, withdraw money, and view account details using EMU8086 Assembly Language. The system stores data in memory variables and registers, demonstrating proper memory management without file handling.

3 Objectives

The major objectives of this project are:

- To develop a functional banking system using EMU8086 Assembly Language
- To understand low-level programming concepts including register manipulation and memory addressing
- To implement account management using memory variables in the data segment
- To perform deposit and withdrawal operations using ADD and SUB instructions
- To maintain customer records in data segment variables with validation
- To implement a PIN-based login system using comparison instructions
- To apply EMU8086 interrupts (INT 21h, INT 10h, INT 16h) for I/O operations
- To demonstrate procedure usage with stack-based register preservation

4 Project Title

Banking Management System Using EMU8086 Assembly Language

This project is specifically designed for the EMU8086 microprocessor emulator, utilizing 16-bit x86 assembly language instructions, DOS interrupts for input/output operations, and memory variables for data storage.

5 Project Scope

5.1 Included Features

The system will include:

- User account creation with name, account number, initial balance, and PIN
- Login system with account number and 4-digit PIN verification
- Deposit money with automatic balance update
- Withdraw money with sufficient balance validation
- Balance inquiry to check current balance
- Display customer details
- Transaction validation using conditional jumps (JE, JNE, JC, JNC)
- Menu-driven interface using loops and procedures

5.2 Limitations

- Console-based interface only (80x25 text mode)
- Single user session (no multi-user concurrency)
- PIN-based security only
- Data lost when program terminates (no file storage)
- Single account management at a time

6 Core Features

6.1 User Features

- User Registration with Account Number and Name
- PIN-based Login Authentication
- Real-time Balance Display
- Deposit Money with Immediate Balance Update
- Withdraw Money with Balance Validation
- Account Details Display
- Menu-driven Navigation

6.2 System-Level Features

- Memory Variable Storage in Data Segment
- Register-based Arithmetic Operations
- PIN Verification using CMP and Conditional Jumps
- Balance Validation before Withdrawal
- ASCII to Binary Conversion for Numeric Input
- Binary to ASCII Conversion for Display
- Stack Operations for Register Preservation (PUSH/POP)
- Procedure-based Modular Code
- DOS Interrupt Handling (INT 21h)

7 Technical Design in EMU8086

7.1 Memory Model

The project follows the standard EMU8086 Assembly Language Memory Model:

1. **Stack Segment (.STACK):** 100h bytes reserved for stack operations
2. **Data Segment (.DATA):** Variables for account storage
3. **Code Segment (.CODE):** All program logic and procedures

7.2 Memory Variables

```
1  .DATA
2      account_no      DW  0
3      user_name       DB  30 DUP('$')
4      balance         DW  0
5      pin_code        DW  0
6      temp_amount     DW  0
7      logged_in       DB  0
8
9      ; Menu Messages
10     welcome_msg      DB  '-----',0DH,0AH,'$'
11     title_msg        DB  '  BANKING MANAGEMENT SYSTEM',0DH,0AH,'$'
12     menu1            DB  '1. Create Account',0DH,0AH,'$'
13     menu2            DB  '2. Login',0DH,0AH,'$'
14     menu3            DB  '3. Deposit Money',0DH,0AH,'$'
15     menu4            DB  '4. Withdraw Money',0DH,0AH,'$'
16     menu5            DB  '5. Check Balance',0DH,0AH,'$'
17     menu6            DB  '6. Display Account Details',0DH,0AH,'$'
18     menu7            DB  '7. Exit',0DH,0AH,'$'
```

7.3 Interrupts Used

- INT 21h AH=01h - Single character input with echo
- INT 21h AH=02h - Display single character
- INT 21h AH=09h - Display string (terminated with \$)

- INT 21h AH=0Ah - Buffered string input
- INT 21h AH=4Ch - Exit program
- INT 10h AH=00h - Set video mode (clear screen)
- INT 16h AH=00h - Keyboard input

7.4 Conversion Techniques

ASCII to Binary Conversion

```

1  ASCII_TO_BINARY  PROC
2      MOV BX, 0
3  CONVERT_LOOP:
4      MOV AH, 01h
5      INT 21h
6      CMP AL, 0Dh
7      JE END_CONVERT
8      SUB AL, 30h
9      MOV AH, 0
10     PUSH AX
11     MOV AX, BX
12     MOV BX, 10
13     MUL BX
14     MOV BX, AX
15     POP AX
16     ADD BX, AX
17     JMP CONVERT_LOOP
18  END_CONVERT:
19     MOV AX, BX
20     RET
21  ASCII_TO_BINARY  ENDP

```

Binary to ASCII Conversion

```

1  BINARY_TO_ASCII  PROC
2      MOV CX, 0
3      MOV BX, 10
4  DIVIDE_LOOP:
5      MOV DX, 0
6      DIV BX
7      PUSH DX
8      INC CX
9      CMP AX, 0
10     JNE DIVIDE_LOOP
11  DISPLAY_LOOP:
12     POP DX
13     ADD DL, 30h
14     MOV AH, 02h
15     INT 21h
16     LOOP DISPLAY_LOOP
17     RET
18  BINARY_TO_ASCII  ENDP

```

8 Environment

The following will be used for development:

Component	Description
Emulator	EMU8086 Microprocessor Emulator
Language	x86 Assembly Language (16-bit MASM syntax)
Architecture	Intel 8086 Microprocessor
Registers	AX, BX, CX, DX, SI, DI, SP, BP
Interrupts	INT 21h, INT 10h, INT 16h
Memory Model	.MODEL SMALL

Table 8.1: Development Environment

9 Methodology

9.1 Phase 1: Requirement Analysis

Understanding banking functionalities and EMU8086 capabilities.

9.2 Phase 2: System Design

Designing menu system and program flow using flowcharts.

9.3 Phase 3: Memory Planning

Allocating variables in data segment for account storage.

9.4 Phase 4: Coding

Implementing features using:

- Procedures (PROC/ENDP) for modularity
- Conditional jumps (JE, JNE, JC, JNC)
- Loops (LOOP instruction)
- Stack operations (PUSH/POP)

9.5 Phase 5: Testing

Testing in EMU8086 emulator using debugger.

9.6 Phase 6: Final Implementation

Running and demonstrating the final project.

10 Program Structure

10.1 Main Menu

1. Create Account
2. Login
3. Deposit Money
4. Withdraw Money
5. Check Balance
6. Display Account Details
7. Exit

10.2 Procedure Structure

- `DISPLAY_MENU` - Shows main menu
- `CREATE_ACCOUNT` - Takes user input, stores in variables
- `LOGIN` - Verifies account number and PIN
- `DEPOSIT_MONEY` - Adds amount to balance
- `WITHDRAW_MONEY` - Subtracts with condition checks
- `SHOW_BALANCE` - Displays current balance
- `DISPLAY_DETAILS` - Shows all account information
- `GET_NUMERIC` - Converts ASCII to binary
- `SHOW_NUMERIC` - Converts binary to ASCII

11 Expected Output

Sample console output:

```
1  =====
2      BANKING MANAGEMENT SYSTEM
3  =====
4  1. Create Account
5  2. Login
6  3. Deposit Money
7  4. Withdraw Money
8  5. Check Balance
9  6. Display Account Details
10 7. Exit
11 =====
12 Enter your choice: 1
13
14 --- CREATE NEW ACCOUNT ---
15 Enter Account Number: 1001
16 Enter Name: Faisal Amin
17 Enter Initial Balance: 5000
18 Enter 4-digit PIN: 1234
19
20 Account Created Successfully!
21
22 =====
23 Enter your choice: 2
24
25 --- USER LOGIN ---
26 Enter Account Number: 1001
27 Enter 4-digit PIN: 1234
28
29 Login Successful!
30
31 =====
32 Enter your choice: 3
33
34 --- DEPOSIT MONEY ---
35 Enter Amount: 2000
36
37 Deposit Successful!
38 Current Balance: 7000
39
40 =====
41 Enter your choice: 7
42
43 Thank you for using Banking Management System!
```

12 Challenges and Solutions

Multi-digit Number Input

Solution: ASCII to binary conversion using multiplication by 10 in a loop.

Displaying Numeric Balance

Solution: Binary to ASCII conversion by repeated division by 10.

String Input for Name

Solution: INT 21h AH=0Ah buffered input.

PIN Validation

Solution: CMP instruction with JE/JNE jumps.

Register Preservation

Solution: PUSH/POP at procedure boundaries.

Overflow Handling

Solution: JC after ADD, JNC after SUB.

13 Learning Outcomes

- EMU8086 Assembly Language syntax and directives
- 16-bit register usage (AX, BX, CX, DX, SI, DI, BP, SP)
- Loops using LOOP instruction
- Conditional jumps (JE, JNE, JC, JNC, JG, JL)
- Procedures with PROC/ENDP
- INT 21h functions for I/O
- ASCII to binary conversion
- Binary to ASCII conversion
- Debugging in EMU8086
- Stack operations (PUSH/POP)
- Memory variable management

14 Future Enhancements

- Multiple account storage using arrays
- Transaction history in memory
- Interest calculation using MUL instruction
- Password masking during input
- Fund transfer between accounts
- Graphical mode using INT 10h

15 Conclusion

This project demonstrates a Banking Management System using EMU8086 Assembly Language. It shows how low-level programming solves real-world problems while providing practical experience with microprocessor architecture.

The system helps students understand banking operations through software while strengthening their understanding of computer organization, microprocessor architecture, and low-level programming in EMU8086.

The successful completion demonstrates mastery of:

- 8086 instruction set and addressing modes
- DOS interrupt services for I/O
- Memory management in assembly
- Arithmetic operations for financial calculations
- Conditional logic for transaction validation
- Modular programming using procedures and stack

References

1. EMU8086 User Manual and Documentation
2. Intel 8086 Microprocessor Programming Guide
3. COAL Course Notes - Bahria University
4. EMU8086 Built-in Help
5. DOS INT 21h Function Reference

Appendix: Code Structure

```
1  .MODEL SMALL
2  .STACK 100h
3
4  .DATA
5      account_no    DW  0
6      user_name     DB  30 DUP(' $ ')
7      balance       DW  0
8      pin_code      DW  0
9      logged_in     DB  0
10
11 .CODE
12 MAIN PROC
13     MOV AX, @DATA
14     MOV DS, AX
15
16     CALL DISPLAY_MENU
17     CALL GET_CHOICE
18
19     MOV AH, 4CH
20     INT 21h
21 MAIN ENDP
22
23 END MAIN
```